EPFC

PRW : Projet de Développement Web
04 - Framework MVC Orienté Objet

EPFC-ULB

Inspiration : bpesquet.developpez.com/tutoriels/php/evoluer-architecture-mvc

Défauts de l'architecture actuelle

EPFC

Mélange entre code HTML/JS (vue) et PHP (traitement, données).

Structure monobloc, code dupliqué, gestion des erreurs éparpillée : **réutilisation, extension et maintenance difficiles**

- Nécessité de diviser le code en fonction de responsabilités claires pour diminuer le couplage et augmenter la cohésion : vues, modèles, actions (contrôleur)

URL difformes et peu lisibles : difficile à utiliser et à indexer par les moteurs de recherche

- Nécessité d'uniformiser. Par exemple, `monsite/members/profile/ben` plutôt que `monsite/profile.php?member=ben`

Configuration en dur dans le code

- Fichier de configuration

Une solution possible : architecture MVC

EPFC

Modèle

Logique métier et accès aux données

Vue

Interactions avec l'utilisateur : présentation, saisie de données, validation JavaScript des formulaires, …

Contrôleur

Dynamique de l'application (les actions)

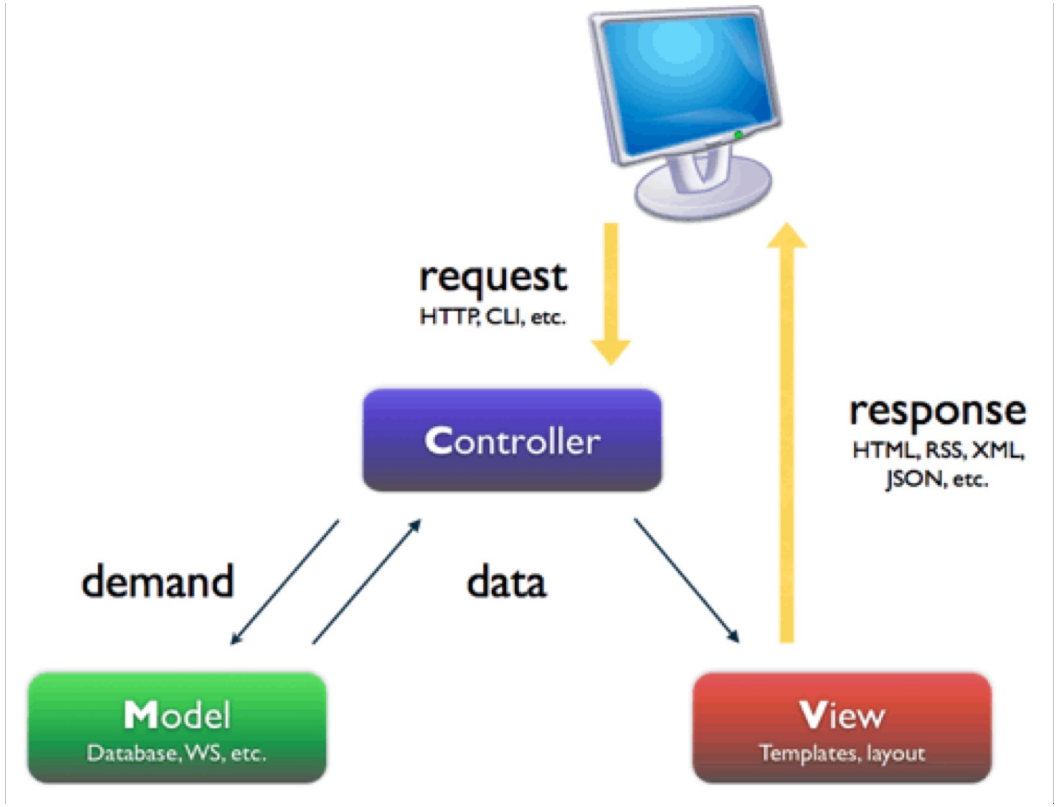
Lien entre l'utilisateur et le reste de l'application.

Fonctionnement du MVC Web

La **demande** de l'utilisateur (requête HTTP) est reçue et **interprétée par le contrôleur**.

Le **contrôleur** utilise les **services du modèle** afin de préparer les **données** à afficher.

Le **contrôleur** fourni ces **données** à la **vue** qui les **présente à l'utilisateur**, par exemple sous forme d'une page HTML ou d'un objet JSON.



Avantages et inconvénients

EPFC

Avantages

Séparation claire des **responsabilités**, couplage faible, forte cohésion.

Division du travail.

Réutilisation possible du modèle et du contrôleur dans d'autres contextes.

Moins de répétitions dans le code.

Les pages HTML (vues) contiennent le moins possible de code serveur (celui-ci est contenu dans le contrôleur et le modèle). Elles sont indépendantes et peuvent par exemple être modifiées par des designers web qui ne sont pas des programmeurs.

Inconvénient

Architecture plus complexe (le prix à payer)**.**

Intérêt d'un framework MVC

EPFC

Un framework (boite à outils) fournit un **ensemble de services de base**, souvent sous forme de classes en interaction.

Cela permet de **libérer le développeur de nombreuses tâches techniques** comme le routage des requêtes, la sécurité, la gestion du cache, …

Il existe plusieurs frameworks MVC PHP sur le marché comme *Symfony*, *Laravel*, *Zend*, ...

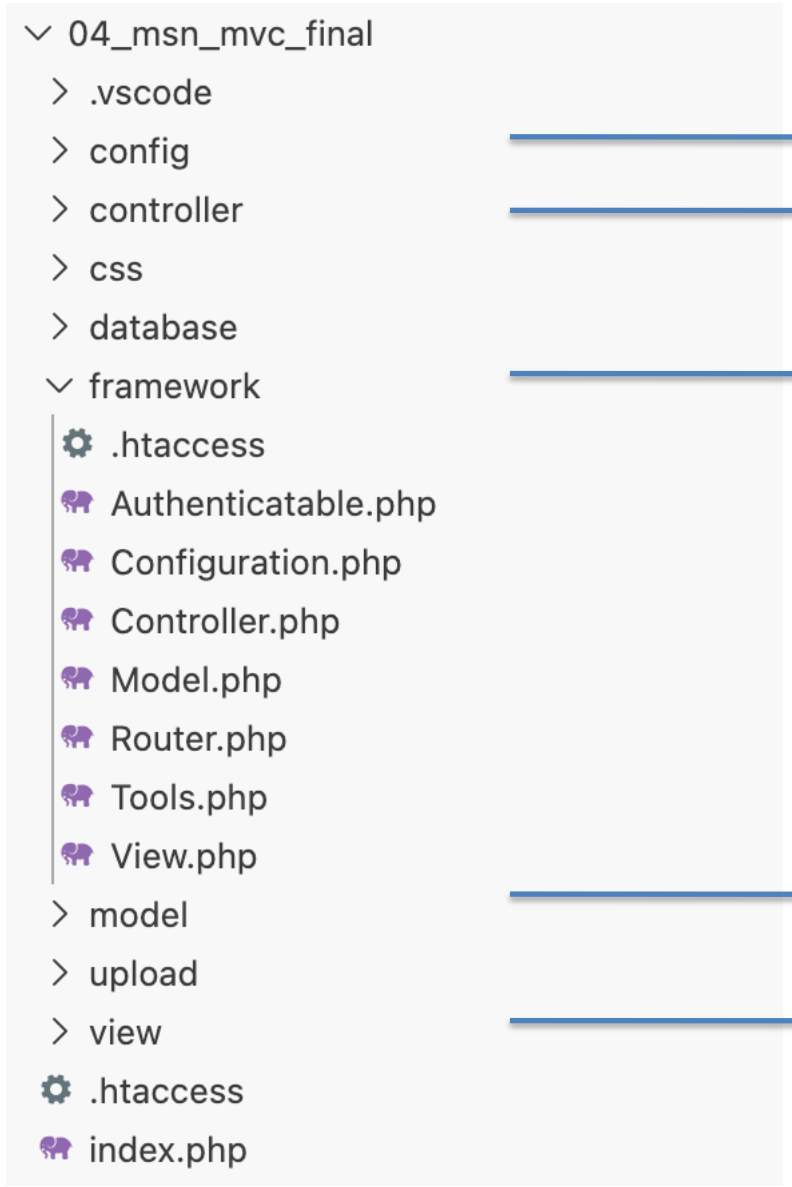
Dans ce cours, nous allons utiliser un **framework "maison" simplifié** afin d'**illustrer** les **concepts principaux** des frameworks MVC.

Présentation du framework MVC

EPFC

1. Organisation des fichiers
2. Gestion des fichiers de configuration
3. La classe Tools : outils génériques
4. Modèle de données OO
5. Gestion des vues
6. Routage des requêtes (routeur ou contrôleur frontal)
7. Contrôleurs
8. Renommage des URLs
9. Récapitulatif

1. Organisation des fichiers



Les fichiers de configuration

Les contrôleurs

Les classes du framework
(communes à chaque application :
les utiliser sans les modifier)

Les modèles

Les vues

Remarquez également d'autres dossiers utiles pour organiser nos fichiers : `css`, `upload`, …

2. Gestion des fichiers de configuration

Le framework s'attend à trouver un fichier `dev.ini` ou `prod.ini` dans le dossier `config`. Le fichier `dev.ini` a priorité sur le fichier `prod.ini`. C'est utile si l'on travaille avec un environnement de développement et un environnement de production.

```
[DB]
dbtype = mysql
dbhost = 127.0.0.1
dbname = my_social_network_base
dbuser = root
dbpassword = root
mysql_path = "C:\PRW2627\tools\xampp\mysql\bin\"

[Controllers]
default_controller = main

[Web]
web_root = "/04_msn_mvc_final/"

...
```

La classe `Configuration` s'occupe de charger ce fichier et de donner accès à ces paramètres. Sans rentrer dans les détails d'implémentation, voici comment obtenir un paramètre :

```
$password = Configuration::get("dbpassword");
```

3. Outils : la classe `Tools`

Certaines fonctionnalités génériques sont implémentées dans la classe `Tools` comme l'échappement et la gestion des erreurs.

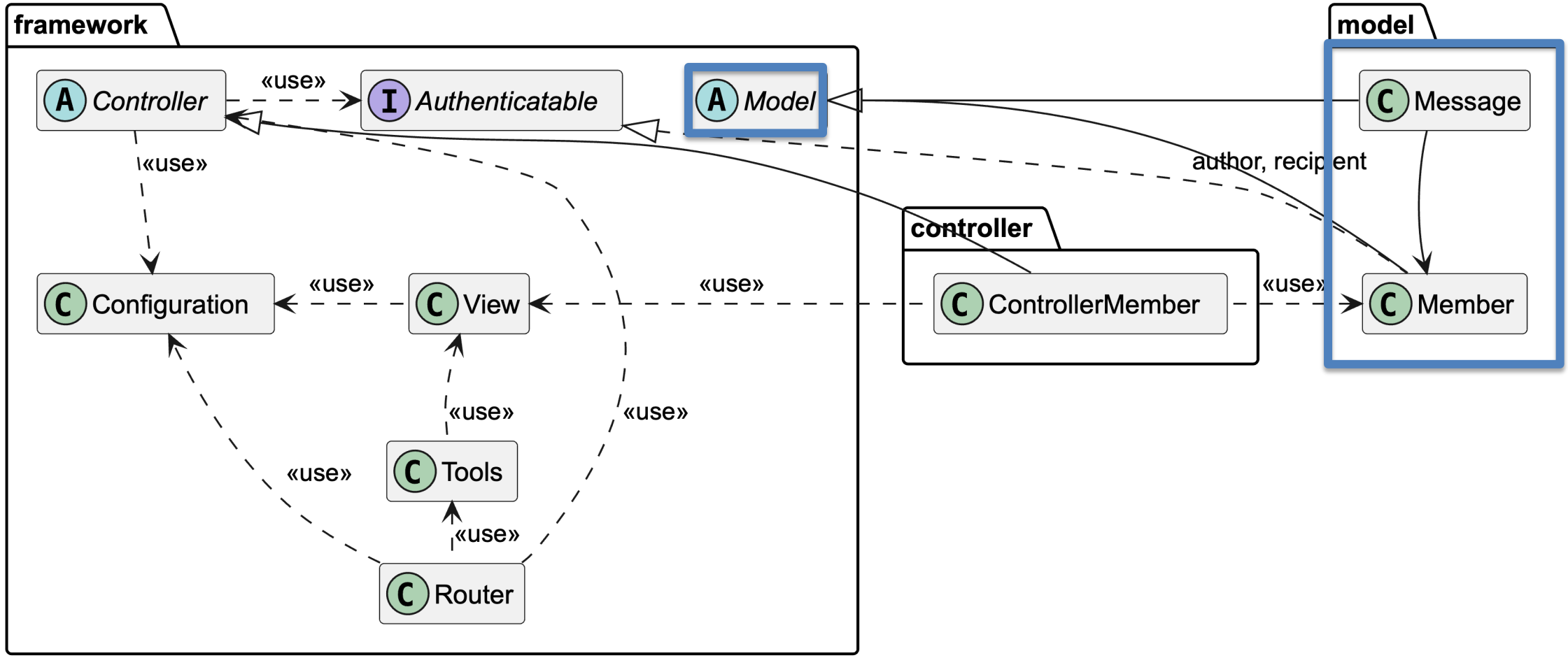
```php
class Tools {

    /** Échappe une valeur pour l'affichage HTML. */
    public static function escape(mixed $var): string {
        return htmlspecialchars((string) $var, ENT_QUOTES, "UTF-8");
    }

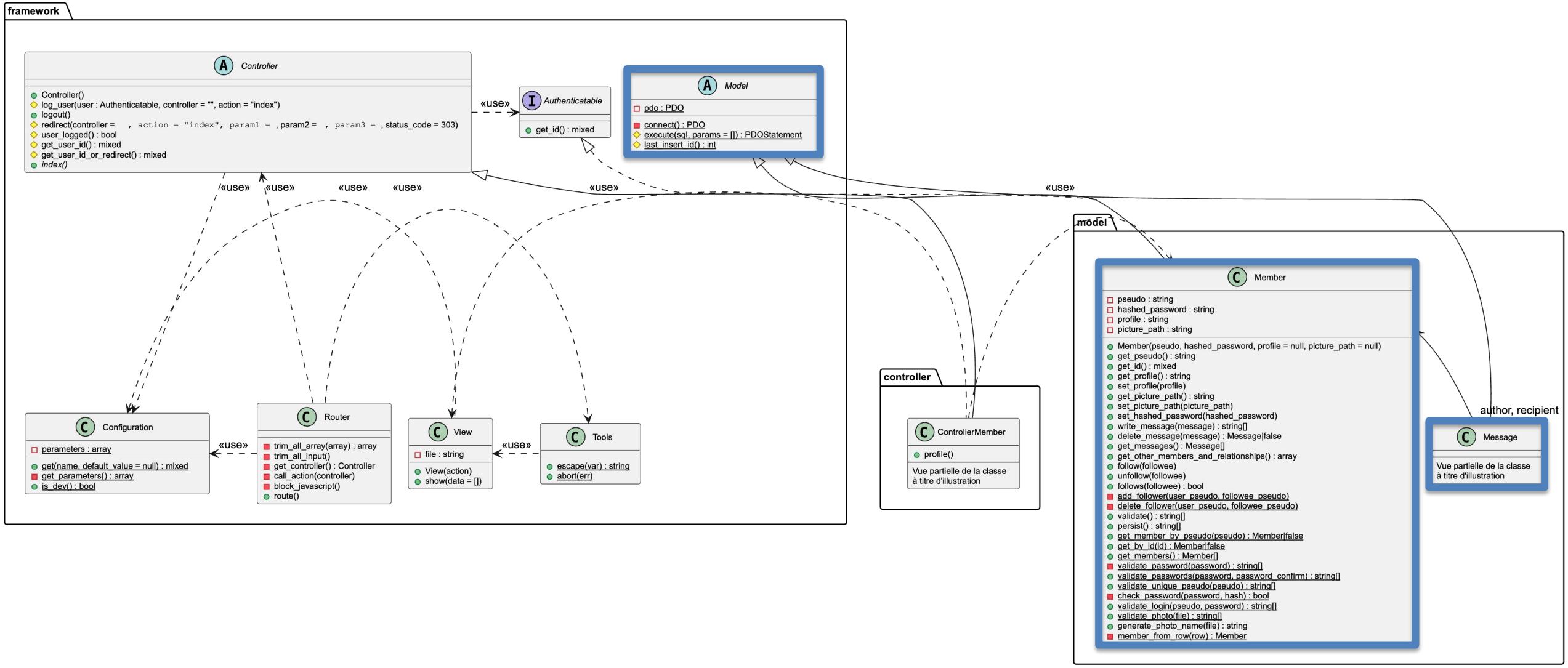
    /** Affiche la page d'erreur en HTTP 500, puis arrête le script. */
    public static function abort(string $err): void {
        http_response_code(500);
        (new View("error"))->show(["error" => $err]);
        die;
    }
}

/** Raccourci de Tools::escape, pour les vues. */
function e(mixed $value): string {
    return Tools::escape($value);
}
```

4. Le modèle de données



4. Le modèle de données



4. Le modèle de données

La classe abstraite `Model` fournit les méthodes nécessaires pour se connecter et interroger la base de données.

Nos modèles devront hériter de cette classe.

```php
abstract class Model {
    private static ?PDO $pdo = null;

    /** Ouvre la connexion PDO, ou renvoie celle déjà ouverte. */
    private static function connect(): PDO {
        try {
            if (self::$pdo === null) {
                $dbtype = Configuration::get("dbtype");
                $dbhost = Configuration::get("dbhost");
                $dbname = Configuration::get("dbname");
                $dsn = "{$dbtype}:host={$dbhost};dbname={$dbname};charset=utf8mb4";
                $dbuser = Configuration::get("dbuser");
                $dbpassword = Configuration::get("dbpassword");

                self::$pdo = new PDO($dsn, $dbuser, $dbpassword);
                self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
                self::$pdo->query("SET sql_mode=(SELECT CONCAT(@@sql_mode,',ONLY_FULL_GROUP_BY'));");
            }
            return self::$pdo;
        } catch (PDOException $e) {
            throw new Exception("PDO connection error : " . $e->getMessage(), previous: $e);
        }
    }

    /** Prépare et exécute une requête, avec les paramètres nommés. */
    protected static function execute(string $sql, array $params = []): PDOStatement {
        try {
            $stmt = self::connect()->prepare($sql);
            $stmt->execute($params);
            return $stmt;
        } catch (PDOException $e) {
            throw new Exception("PDO query error : " . $e->getMessage(), previous: $e);
        }
    }

    /** Renvoie l'identifiant de la dernière insertion. */
    protected static function last_insert_id(): int {
        return (int) self::connect()->lastInsertId();
    }
}
```

4. Implémentation du modèle de données : Member

EPFC

```php
class Member extends Model implements Authenticatable {
    public function __construct(
        private string $pseudo,
        private string $hashed_password,
        private ?string $profile = null,
        private ?string $picture_path = null
    ) {}

    // getteurs et setteurs (passés) ...

    public function write_message(Message $message): array ...
    public function delete_message(Message $message): Message|false ...
    public function get_messages(): array ...
    public function get_other_members_and_relationships(): array ...
    public function follow(Member $followee): void ...
    public function unfollow(Member $followee): void ...
    public function follows(Member $followee): bool ...
    public function validate(): array ...
    public function persist(): array ...
    public function generate_photo_name(array $file): string ...

    public static function get_member_by_pseudo(string $pseudo): Member|false ...
    public static function get_by_id(mixed $id): Member|false ...

    private static function member_from_row(array $row): Member {
        return new Member($row["pseudo"], $row["password"],
                          $row["profile"], $row["picture_path"]);
    }

    public static function get_members(): array {
        $stmt = self::execute("SELECT * FROM Members ORDER BY pseudo ASC");
        $results = [];
        foreach ($stmt->fetchAll() as $row) {
            $results[] = self::member_from_row($row);
        }
        return $results;
    }
    ...
}
```

La classe `Member` étend la classe `Model`

Un membre est un objet qui possède 4 attributs.

Une série de méthodes d'instance permettent de manipuler l'objet courant.

Une série de méthodes statiques permettent d'interroger ou de modifier la base de données.

Attention, les membres sont maintenant des objets. Ce ne sont plus des tableaux associatifs.

4. Implémentation du modèle de données : Message

EPFC

```php
class Message extends Model {
    public function __construct(
        private Member $author,
        private Member $recipient,
        private string $body,
        private bool $private,
        private ?int $post_id = null,
        private ?string $date_time = null
    ) { }

    // getteurs et setteurs (passés) ...

    public function is_visible_to(Member $viewer): bool ...
    public function can_be_deleted_by(Member $member): bool ...
    public function validate(): array ...
    public function persist(): ...
    public function delete(Member $member): Message|false ...

    public static function get_messages(Member $member): array {
        $stmt = self::execute(
            "SELECT * FROM Messages WHERE recipient = :pseudo ORDER BY date_time DESC",
            ["pseudo" => $member->get_pseudo()]
        );
        $messages = [];
        foreach ($stmt->fetchAll() as $row) {
            $messages[] = self::message_from_row($row);
        }
        return $messages;
    }

    public static function get_message(int $post_id): Message|false ...
    private static function message_from_row(array $row): Message ...
}
```

Même remarque : les messages sont maintenant des objets

5. Gestion des vues (*templates*)

Les vues produisent une page HTML selon les données reçues, sans les modifier.

Elles sont nommées "`view_<nom>.php`" et sont rangées dans le dossier `view`.

Exemple : `view_profile.php`

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title><?= e($member->get_pseudo()) ?>'s Profile!</title>
        <base href="<?= $web_root ?>"/>
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <link href="css/styles.css" rel="stylesheet" type="text/css"/>
    </head>
    <body>
        <div class="title"><?= e($member->get_pseudo()) ?>'s Profile!</div>
        <?php include("menu.html"); ?>
        <div class="main">
            <?php if (mb_strlen($member->get_profile() ?? "") == 0): ?>
                No profile string entered yet!
            <?php else: ?>
                <?php /* nl2br après e() pour les retours à la ligne du profil */ ?>
                <?= nl2br(e($member->get_profile()), false) ?>
            <?php endif; ?>
            <br><br>
            <?php if (mb_strlen($member->get_picture_path() ?? "") == 0): ?>
                No picture loaded yet!
            <?php else: ?>
                <img src="upload/<?= e($member->get_picture_path()) ?>" width="100" alt="<?= e($member->get_pseudo()) ?>'s photo!">
            <?php endif; ?>
            <br><br>
            <a href="message/index/<?= e($member->get_pseudo()) ?>">View <?= e($member->get_pseudo()) ?>'s messages</a>
        </div>
    </body>
</html>
```

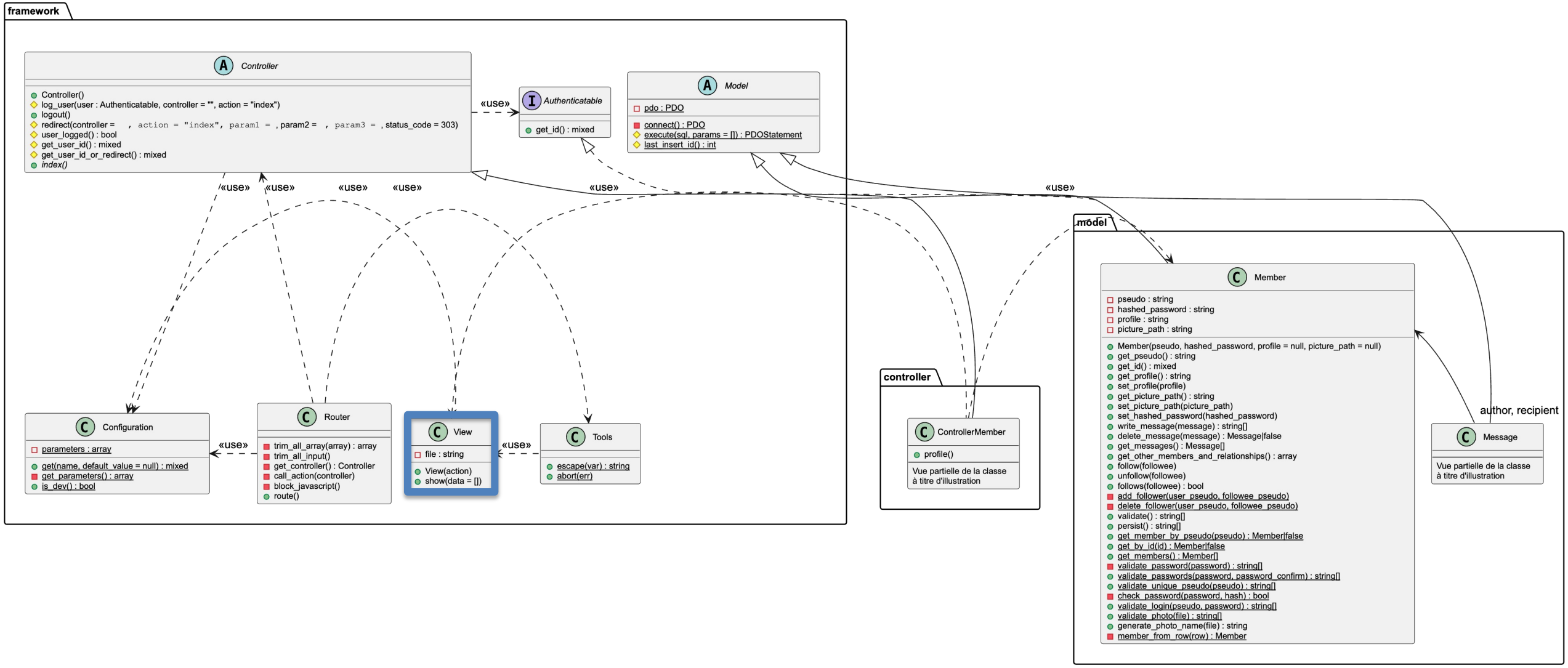
La fonction `e()` vient de `Tools` et est un raccourci pour la fonction `escape()`

`<?= expr ?>`
est une notation réduite pour
`<?php echo expr; ?>`

Constatez que nous avons utilisé une autre syntaxe, plus claire, pour exprimer les blocs.

Remarquez que le code PHP est très réduit et ne modifie rien.

5. La classe `View`



5. La classe `View`

Le constructeur de la classe `View` reçoit un nom et va chercher le fichier associé (`view/view_<nom>.php`).

La méthode `show()` reçoit un tableau associatif qui contient les données à fournir à la vue. Elle transforme ce tableau (`extract()`) en un ensemble de variables attendues par la vue et renvoie la vue au client.
Elle ajoute toujours `$web_root` car il est nécessaire dans toutes les vues.

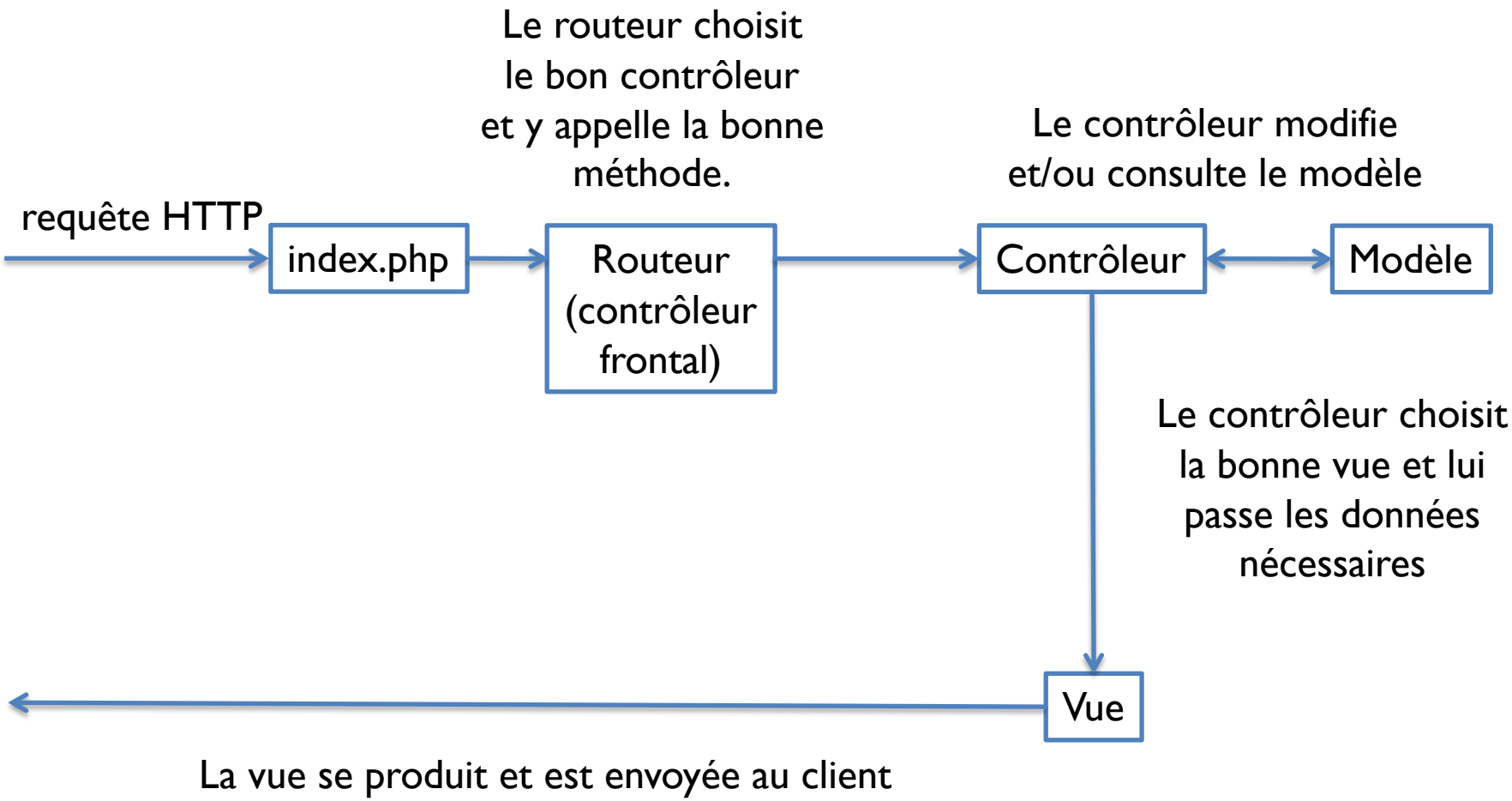
Exemple :

```
$m = Member::get_member_by_pseudo("ben");

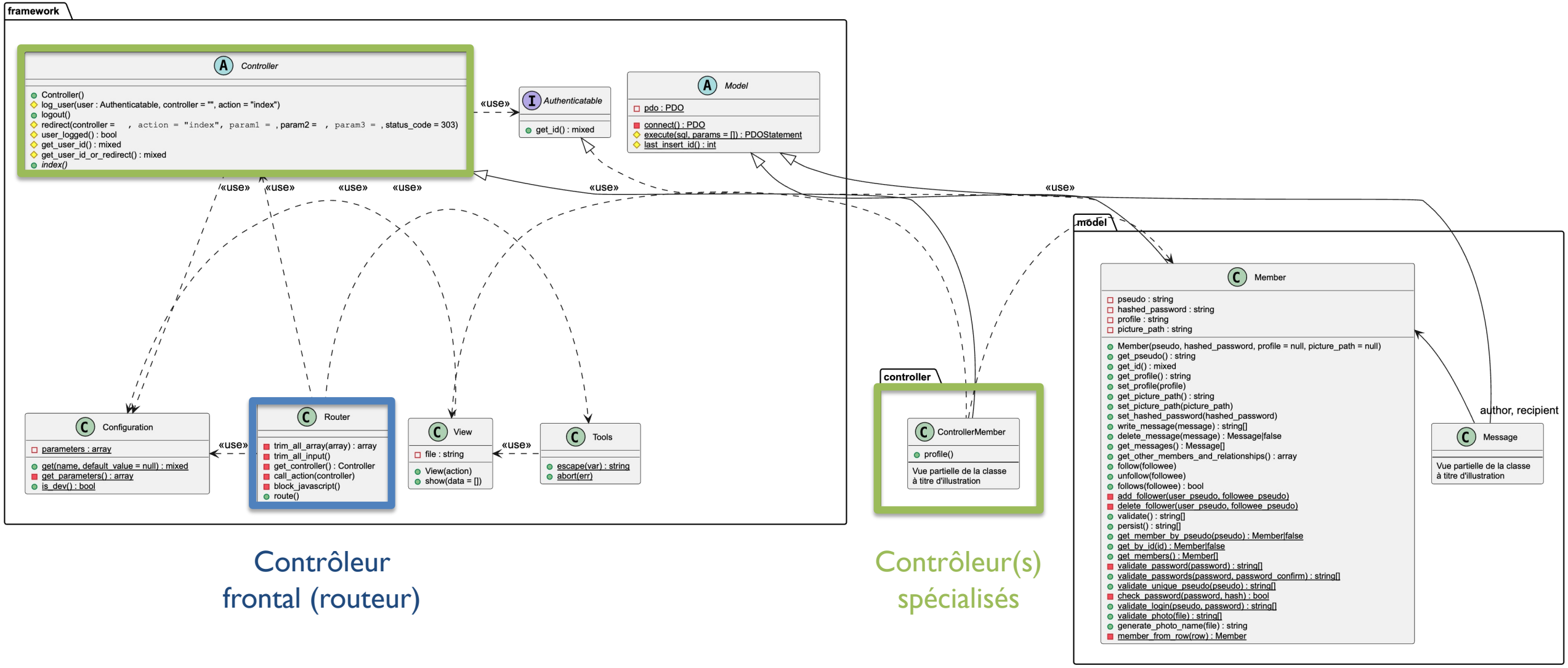
$v = new View("profile");
$v->show(["member" => $m]);

// ou (new View("profile"))->show(["member" => $m]);
```

6. Routage des requêtes : chemin complet



6. Routage des requêtes



Contrôleur frontal (routeur)

Contrôleur(s) spécialisés

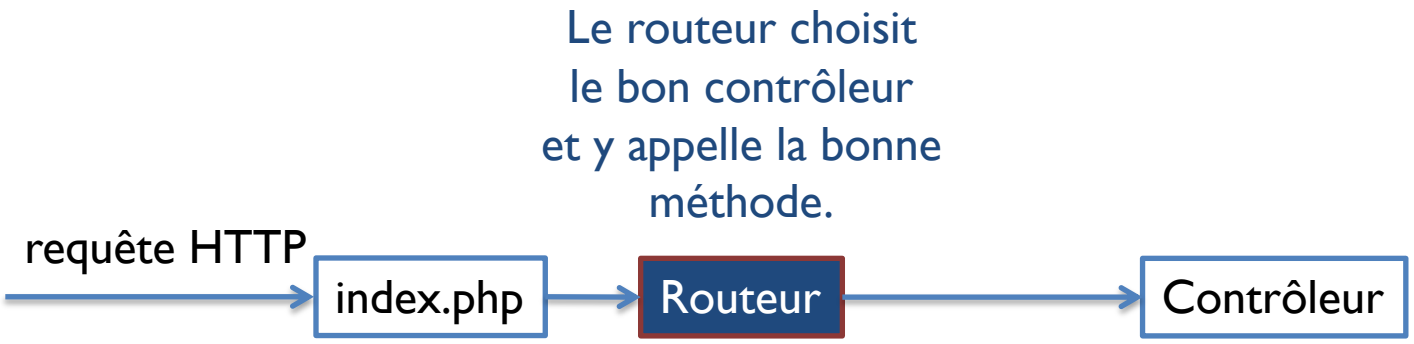
6. Routage des requêtes (contrôleur frontal)

EPFC

Buts

- Uniformiser les URLs
- Ne pas exposer la structure du site (fichiers php)
- Rassembler les politiques communes (exceptions, page d'erreur, *trim*, ...)

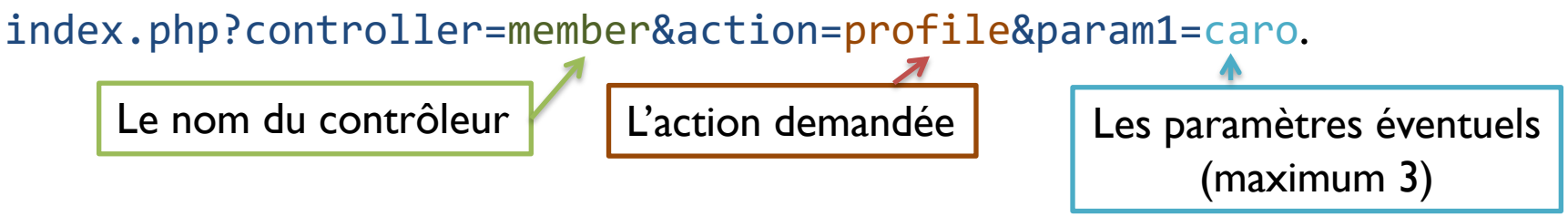
Le contrôleur frontal (le routeur) reçoit la requête du client et la transmet à un contrôleur spécialisé qu'il instancie (gestion des membres, gestion des messages, gestion du login, ...). L'instance du contrôleur ne vit que pour l'exécution d'une requête.



6. Fonctionnement du routeur.



La page `index.php` reçoit une requête HTTP uniformisée de la forme



La requête est transmise au routeur qui l'analyse, la nettoie (*sanitize*), crée une instance du contrôleur spécialisé (ici `ControllerMember`) et passe la requête à la méthode demandée (ici la méthode `profile()`).

Remarques :

- Si l'action n'est pas précisée, on appelle l'action par défaut (la méthode `index()`).
- Si le contrôleur n'est pas précisé, le contrôleur par défaut indiqué dans le fichier de configuration est utilisé.
- Notez que l'uniformisation des URL est indispensable pour que cela fonctionne.
- Si une exception est déclenchée, le routeur la rattrape et produit une vue (page HTML). La gestion des erreurs est donc concentrée ici.

6. La classe Router



EPFC

```php
class Router {
    private function trim_all_array(array $array): array ...
    private function trim_all_input(): void ...
    private function get_controller(): Controller ...
    private function call_action(Controller $controller): void ...
    private function block_javascript(): void ...

    public function route(): void {
        try {
            $this->trim_all_input();
            for ($i = 1; $i <= 3; ++$i) {
                if (($_GET["param$i"] ?? null) === "") {
                    unset($_GET["param$i"]);
                }
            }
            $controller = $this->get_controller();
            $this->block_javascript();
            $this->call_action($controller);
        } catch (Throwable $ex) {
            if (Configuration::is_dev()) {
                Tools::abort($ex->getMessage() . "\n" . $ex->getTraceAsString());
            } else {
                Tools::abort($ex->getMessage());
            }
        }
    }
}
```

index.php

```php
date_default_timezone_set('Europe/Brussels');

error_reporting(E_ALL);
ini_set('display_errors', 1);
ini_set('default_charset', "UTF-8");

require_once 'framework/Router.php';
require_once 'framework/Configuration.php';

if (!Configuration::is_dev()) {
    ini_set('display_errors', 0);
}

(new Router())->route();
```

7. Les contrôleurs spécialisés

Chaque contrôleur spécialisé hérite de la classe abstraite `Controller`. Celle-ci contient les méthodes communes à tout contrôleur : gestion des sessions, connection d'un utilisateur, redirection, …

```
abstract class Controller {
    public function __construct() ...
    protected function log_user(Authenticatable $user, string $controller = "", string $action = "index"): void ...
    public function logout(): void ...
    protected function redirect(string $controller = "", string $action = "index", string $param1 = "", string $param2 = "",
                                string $param3 = "", int $status_code = 303): void ...
    protected function user_logged(): bool ...
    protected function get_user_id(): mixed ...
    protected function get_user_id_or_redirect(): mixed ...
    public abstract function index(): void;
}
```

Remarquez la méthode abstraite `index()` : chaque contrôleur devra implémenter au moins cette méthode : il s'agit de l'action par défaut du contrôleur.

```
/** Objet qu'on peut connecter : fournit la clé à stocker en session. */
interface Authenticatable {

    /** Clé primaire à mémoriser en session (int, string, …). */
    public function get_id(): mixed;
}
```

7. Implémentation d'un contrôleur

Les contrôleurs définissent comme méthodes publiques les différentes actions possibles (celles que le routeur pourra appeler).

Ces méthodes modifient au besoin le modèle, récupèrent les données nécessaires qu'elles passent ensuite à une vue.

```php
class ControllerMember extends Controller {

    public function index(): void {
        $this->profile();
    }

    public function profile(): void {
        $member = Member::get_by_id($this->get_user_id_or_redirect());
        $pseudo = $_GET["param1"] ?? "";
        if ($pseudo !== "") {
            $found = Member::get_member_by_pseudo($pseudo);
            if ($found === false) {
                throw new Exception("Unknown member");
            }
            $member = $found;
        }
        (new View("profile"))->show(["member" => $member]);
    }

    public function edit_profile(): void ...
}
```

Attention au nom de la classe.
Pour que le routeur puisse la retrouver, son nom commence par `Controller` et est suivi de son nom avec une majuscule.

La méthode `profile()` sera exécutée par le routeur s'il reçoit la requête `index.php?controller=member&action=profile`

8. Renommage des URLs

EPFC

Pour terminer, nous voulons que les URL soient lisibles afin, par exemple, d'être indexées par les moteurs de recherche.

Plutôt que http://monsite.be/index.php?controller=member&action=profile&param1=caro, nous voulons http://monsite.be/member/profile/caro.

Cela peut se faire via le renommage d'URL (*URL rewriting*). A l'aide du fichier `.htaccess` suivant placé à la racine du site web, le serveur web Apache renomme les urls comme désiré.

```
RewriteEngine on
RewriteBase /04_msn_mvc_final/
RewriteRule ^([\w\_\-\%\~\+]*)/?([\w\_\-\%\~\+]*)?/?([\w\_\-\%\~\+]*)?/?([\w\_\-\%\~\+]*)?/?([\w\_\-\%\~\+]*)?/?$
index.php?controller=$1&action=$2&param1=$3&param2=$4&param3=$5 [NC,L]
```

Des conditions sont nécessaires pour que cela fonctionne bien : chaque page HTML doit mentionner son URL de base (le chemin relatif par rapport au nom de domaine géré par le serveur web) via l'élément `<base>` du `<head>` et la même URL doit être mentionnée dans le fichier `.htaccess` (`RewriteBase`).

9. Récapitulatif

Le navigateur envoie la requête `/member/profile/caro`

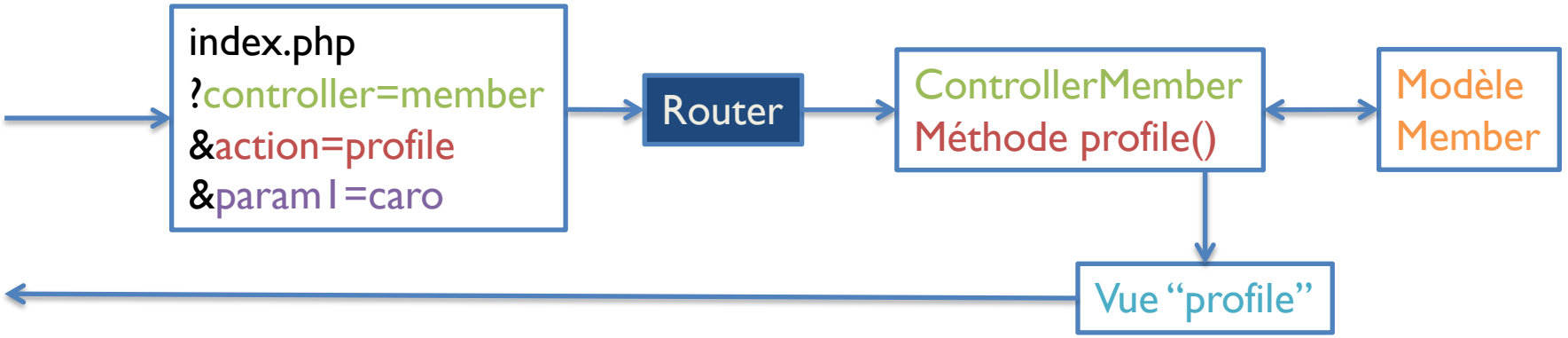
Le serveur web renomme cette URL en `index.php?controller=member&action=profile&param1=caro`.

La page `index.php` instancie le routeur et lui transmet la requête.

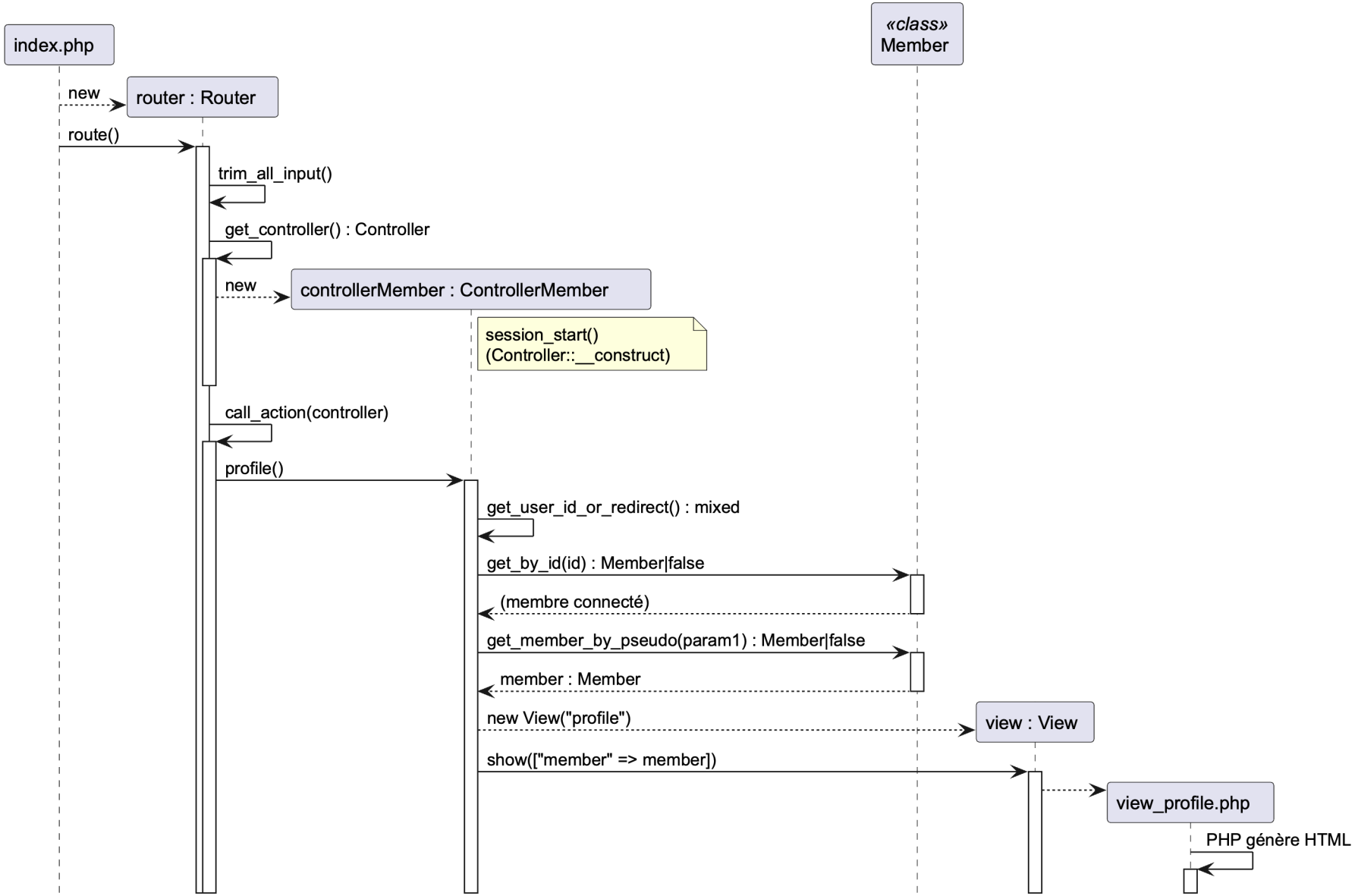
Le routeur nettoie la requête, crée une instance de `ControllerMember` et y appelle la méthode `profile()` en lui transmettant la requête.

La méthode `profile()` récupère le paramètre `param1` et demande à la classe `Member` le membre dont le pseudo est "`caro`".

Cette même méthode crée la vue "`profile`" et lui demande de s'afficher après lui avoir passé l'objet concernant le membre "`caro`".



9. Récapitulatif : diagramme de séquence



9. Récapitulatif : modification du profil

EPFC

L'URL appellée est /member/edit_profile.

Le routeur appelle la méthode edit_profile() sur un ControllerMember.

```php
class ControllerMember extends Controller {
    public function edit_profile(): void {
        $member = Member::get_by_id($this->get_user_id_or_redirect());
        $errors = [];

        $profile = $_POST["profile"] ?? null;

        if ($profile !== null) ...

        (new View("edit_profile"))->show(["member" => $member, "errors" => $errors]);
    }
    ...
}
```

Si l'utilisateur n'est pas connecté, on le redirige vers l'index.

Gestion de l'upload de l'image et de la modification du profil (via le modèle).

Production de la vue "edit_profile" en lui passant le membre en question et les messages d'erreurs éventuels.